Project brief: everything you need to present this

Fine-Tuning Neural Machine Translation for Kenyan Public Service Announcements — adding Ekegusii to NLLB-200.

TEAM

Weldesenbet Zeray · Samuel Abrha · Hetal Kumbharana · Halima Mohammed · Peter Kidiga · Mitchelle Moraa

SUPERVISOR

Prof. Edward Ombui

United States International University–Africa

Natural Language Processing · School of Science and Technology · 2026

2. Why this is hard, in three facts

Ekegusii is absent, not merely poorly supported. This distinction matters and examiners will probe it. A language NLLB supports badly can be improved with fine-tuning alone. A language it does not support has no vocabulary entry, no embedding, no way to signal "translate into this." The first technical problem was not quality — it was existence.

There was no corpus. No published English–Ekegusii parallel dataset of usable size. Everything had to be assembled: scripture, storybooks, scraped sentences, and a PSA corpus supplied by the supervisor. After quality filtering this comes to roughly 35,000 aligned records, which yield **62,669 unique training pairs**. For comparison, high-resource language pairs train on tens of millions.

The available data is the wrong genre. Most of what exists in Ekegusii is scripture. A model trained on scripture learns the language but not the voice of a government notice. We measured this before training: **53.6% of English content-word types in the PSA corpus (42.3% of tokens) never appear anywhere in the aligned training data** — portal, bursary, HELB, KUCCPS, iTax, Huduma. That number is the project's central risk and it drove every design decision after it.

3. What we built, step by step

3.1 The corpus

SOURCE	PAIRS	LANGUAGES	NOTES
Ekegusii Revised Bible	30,753	en · guz · swh	Verse-aligned triples. English from the Berean Standard Bible, Kiswahili from Neno: Bibilia Takatifu
African Storybook	110	en · guz · swh	Contemporary narrative prose
Lughayangu	316	en · guz	Everyday sentences, scraped from 364 pages
Kenyan PSA corpus	4,082	en · guz · swh	Real public notices, supervisor-supplied

Know your three numbers. These get confused, and an examiner who spots the confusion will press on it:

COUNT	NUMBER	WHAT IT MEANS
Aligned records	~35,300	Rows in the corpus — a Bible verse, a storybook line, a PSA
Unique training pairs	**62,669**	Each record gives eng→guz, and swh→guz where Kiswahili exists. **Quote this one**
Training examples per epoch	79,745	The above with the 5,692 PSA pairs repeated 4× to weight them

Upsampling the PSA data fourfold is why the third number is larger. It is a legitimate training choice, but it is not "we had 80,000 sentences" — say **62,669 pairs**, or "more than 60,000", and you cannot be caught out.

A source that contributed nothing. We also pulled 27,575 English–Ekegusii pairs from 4laws. Every one turned out to be a duplicate of the eBible text we already had. We documented that rather than quietly dropping it — a source that adds zero rows is a finding about the state of Ekegusii resources.

3.2 Quality control, and why it is not boilerplate

We trained a **character-trigram Naive Bayes language identifier on the project's own corpus** rather than using a general-purpose one, because off-the-shelf identifiers do not know Ekegusii. It reached 99.5–100% accuracy separating English, Ekegusii and Kiswahili.

It then caught a problem in our own output: 11 of 121 African Storybook rows were not Ekegusii at all but licence and credits text (`* License: [CC-BY] * Text: ...`) that the scraper had swept up. **A human review had passed those rows.** That is the argument for automated verification, and it is a good story to tell.

Other gates: mojibake repair with `ftfy` applied to a fixed point, whole-span deletion of footnotes and cross-references during USFM parsing, rejection of rows where the Ekegusii and English are identical, and rejection of ambiguous alignments where one Ekegusii string maps to dissimilar English.

One deliberate asymmetry. Long document extracts and length-ratio outliers were *kept in training* and *excluded from the test set*. They are still Ekegusii and still teach the language, but scoring on a 200-word extract would measure document handling rather than PSA translation.

3.3 Adding the language

1. `guz_Latn` is appended to the tokenizer as a special token; the embedding matrix grows from 256,204 to 256,205 rows.
2. The new row is **copied from `kik_Latn` (Kikuyu)** plus 1% Gaussian noise, so the two can diverge during training.
3. Because the tokenizer's language machinery does not know about a language added after pretraining, input sequences are built by hand as `[src_lang] tokens [eos]`, with `forced_bos_token_id` selecting the target.

This is transfer learning, and the seed is where the transfer happens. Transfer learning means reusing what a model already knows rather than starting over. NLLB has already learned Bantu structure from Kikuyu, Kiswahili and others; seeding `guz_Latn` from `kik_Latn` hands Ekegusii that knowledge as a starting point. A random embedding would give the decoder no prior at all, and 62,669 pairs is nowhere near enough to learn a language from nothing.

If you are asked to name your method in one phrase, it is this: **transfer learning to add an unsupported language to a pretrained multilingual translation model, followed by domain adaptation to PSA register.**

3.4 The experiment

Three training runs, identical initialisation, data, hyperparameters and random seed. **Only the ordering differs.**

RUN	STARTS FROM	TRAINS ON	PURPOSE
Stage 1	tokenizer-extended base	56,977 general examples	learn Ekegusii — the baseline
Stage 2	**stage 1's weights**	30,357: PSA ×4 + 25% scripture replay	adapt to PSA register
Mixed	tokenizer-extended base	79,745 examples — everything, one pass	control: did ordering help?

The hypothesis was that stage 1 → stage 2 would beat mixed. **Replay** exists because continuing training on a narrow new domain normally causes *catastrophic forgetting* — the model gets better at PSAs and worse at everything else. Mixing 25% of the old data back in is the standard defence.

Setup: `facebook/nllb-200-distilled-600M`, full fine-tune, 8-bit AdamW, bf16, gradient checkpointing, 3 epochs per run, lr 5e-5 (stage 1 and mixed) and 1.5e-5 (stage 2), beam 4 at decode, seed 42, on one A100-80GB.

---

4. Why chrF2++ and not BLEU

This will be asked. The answer is morphology.

BLEU counts matching word n-grams. It asks: is this exact word, and this exact sequence of words, present in the reference?

Ekegusii is agglutinative. Subject, tense, object and negation attach to a verb stem, so a single Ekegusii word can carry what English spreads across five. Get the stem right and one affix wrong and BLEU scores that entire word as a miss. A translation a fluent speaker would call *almost right* scores close to zero.

chrF2++ counts character n-grams, plus word unigrams and bigrams. A correct stem with an imperfect affix earns most of the credit it deserves — which is roughly what a human rater would give it.

This is the standard choice, not a convenient one. chrF (Popović, 2015) and chrF++ (Popović, 2017) were designed for this problem and correlate better with human judgement on morphologically rich and low-resource languages. Both are computed with sacreBLEU. **We report BLEU alongside** so the numbers stay comparable with other work.

Why not COMET? COMET is a learned metric — a neural model that scores translations. Its encoder has never seen Ekegusii, so its scores here would be noise dressed up as precision. Saying this shows you understand the metric rather than skipping it.

Our own numbers make the argument

English into Ekegusii, real PSAs:

SYSTEM	BLEU	CHRF2++
Stock NLLB-200	1.63	14.56
Stage 1, language only	2.54	23.96
Two-stage curriculum	7.21	34.93

Stage 1 has learned Ekegusii — it scores 49.66 chrF2++ on scripture — yet BLEU puts it at **2.54** on PSAs, barely above a model that cannot produce the language at all. A metric that cannot separate a working system from a broken one is not measuring the right thing for this language pair. Quote these two columns if you are challenged on the metric choice; they are more persuasive than the theory.

Every number is reported with bootstrap confidence intervals. A two-point difference with no interval is not a result.

5. Results

chrF2++, higher is better. Four systems, identical test sets.

DIRECTION	TEST SET	N	STOCK NLLB*	STAGE 1	TWO-STAGE	SINGLE PASS
eng→guz	Real PSAs	570	14.56	23.96	34.93	**40.97**
eng→guz	Scripture (held out)	1,459	14.88	49.66	48.95	**49.81**
eng→guz	Everyday prose	200	11.41	28.97	29.37	**32.98**
swh→guz	Real PSAs	371	14.13	24.49	34.50	**39.61**
swh→guz	Scripture (held out)	1,463	14.65	49.11	48.80	**49.30**

* Stock NLLB cannot produce Ekegusii. It is asked for Kikuyu — the nearest language it supports — purely to establish what "no Ekegusii support" looks like numerically. **It is a floor, not a fair baseline**, and you should say so before anyone else does.

Both directions agree, which matters more than it looks: the gain is not an artifact of English being the source. Kiswahili into Ekegusii on real PSAs goes 14.13 → 39.61, a 180% relative gain against English's 181%; and the released model beats the curriculum by 5.11 there against 6.04 for English.

One asymmetry to declare before you are asked. There is no everyday-prose test set for Kiswahili. The Lughayangu corpus is English–Ekegusii only, so that third column exists for one direction and not the other. That is a limit of the available data, not a choice we made.

The headline numbers:

- **+26.41 chrF2++ over the floor** on real PSAs — a 181% relative gain
- **+17.01 over stage 1** — this is the value of PSA adaptation specifically
- **Zero forgetting.** The released model scores 49.81 on scripture, *higher* than the model trained only on scripture

6. The finding: the curriculum lost

The two-stage curriculum scored **34.93** on real PSAs. Training on the same data in a single pass scored **40.97**. Curriculum benefit is **−6.04 chrF2++**.

Two obvious objections, both ruled out:

"The single pass just saw more data." It did not. It sees every unique example the curriculum sees. The replay slice is duplicated scripture already present in stage 1, not new material.

"The single pass got more compute." The opposite. At equal epoch counts the two-stage run takes roughly **9% more gradient updates**, because replayed rows are trained on twice. It had more compute and still lost.

A plausible mechanism. Stage 2's dev loss bottomed at 1.357 and drifted back to 1.417 — it overfitted a small, PSA-heavy set. The single pass sees PSA examples interleaved with scripture throughout, which regularises it. And the forgetting that replay was designed to prevent never arises when the data is not split in the first place.

How to present this

Do not apologise for it. Say it plainly:

> *"We hypothesised that a two-stage curriculum would outperform joint training. We built the control that could disprove that, and it did — by six chrF2++. We release the control. The reason our headline number is worth believing is precisely that we ran the experiment that could have embarrassed us."*

A control that falsifies your hypothesis is a **result**. A project that only reports the comparison it hoped for is the weaker piece of work.

---

7. Verification — how we know the numbers are real

Three checks worth mentioning if you are asked about rigour:

Test-set leakage. The Bible contains 421 duplicate English verses — parallel passages where Chronicles repeats Kings. A verse could land in test while an identical copy sat in training. We deduplicate on `(source language, target language, lowercased source)` and dropped **67 test rows and 28 dev rows**, reporting the count rather than hiding it.

Model identity. We verified by **weight fingerprint** that each published checkpoint is the one we evaluated: cosine similarity between each Hugging Face repository and its local directory is exactly 1.000000, while every cross-pair sits at 0.99985–0.999985. Filenames prove nothing; the tensors do.

Behavioural confirmation. Re-scoring 150 held-out PSA rows with each downloaded model reproduced the ordering and the approximate values — 23.77 / 36.53 / 43.55 against reported 23.96 / 34.93 / 40.97.

---

8. Limitations — say these before you are asked

No human evaluation. Every number is an automatic metric against a single reference. No fluency, adequacy or cultural-accuracy ratings from Ekegusii speakers. This is the largest outstanding gap, and the demo's correction form exists to close it.

Unverified references. The PSA Ekegusii was supplied by the supervisor. The translator and the quality-assurance process are unknown; no inter-annotator agreement exists.

Scripture-heavy mixture. About 57,000 of the 62,669 unique pairs are Bible verses. The model may still lean formal on unfamiliar input.

Institutional vocabulary. The 53.6% content-word gap was *measured, not solved*. HELB, KUCCPS and iTax remain the weakest cases.

The confidence score is not calibrated. It reports the model's own certainty — the geometric-mean per-token probability — not a probability of being correct. Neural MT is routinely fluent, confident and wrong.

Licensing. The weights derive from the Ekegusii Revised Bible, © Bible Society of Kenya. The repositories are private pending clearance.

Not suitable for medical, legal or emergency communication without review by a fluent Ekegusii speaker.

---

9. Questions you will be asked, with answers

Why NLLB and not train from scratch? Thirty-five thousand sentence pairs cannot train a translation model from scratch — you need millions. NLLB brings a multilingual prior: it already knows Kiswahili, Kikuyu and Bantu structure generally, so we are adapting existing knowledge rather than creating it. This is transfer learning, and it is the only viable approach at this data scale.

Why the 600M model and not 1.3B? Three full fine-tuning runs were needed for the ablation, on a shared GPU. The 600M distilled model made three runs affordable; 1.3B would have made one. Given the choice between a bigger model with no control and a smaller model with a proper experiment, the experiment is worth more.

Why Kikuyu for the embedding initialisation? It is the nearest Kenyan Bantu language NLLB supports. It is not the nearest genetically — Ekegusii and Kikuyu are in different Bantu subgroups — so the honest framing is "best available proxy," not "closest relative." It shares noun-class morphology and much of the orthography, which is what matters for initialisation.

Isn't training mostly on the Bible a serious bias? Yes, and we measured it rather than hoping. That measurement — the 53.6% vocabulary gap — is why the PSA data exists in the mixture and why the register question is the centre of the project rather than a footnote.

How do you know your test set is clean? Explicit deduplication against the training keys, which dropped 67 test and 28 dev rows caused by 421 duplicate Bible verses. The check is an assertion in the data-building notebook, so it runs every time the splits are rebuilt.

Your curriculum hypothesis failed. Is this project a failure? No. The curriculum was one hypothesis; the project's objective was a working Ekegusii translator, and we have one at +181% over the floor. The failed hypothesis is a contribution: it is evidence that for a corpus this size, joint training beats staged training — and we can show *why* it is not a data or compute artifact.

Could the mixed model simply be better because it saw more? No. Same unique examples, and roughly 9% *fewer* gradient updates. Both alternative explanations are ruled out by construction.

How do we know the published models are the ones you evaluated? Weight fingerprinting — cosine similarity in float64 between each published repository and the local checkpoint, plus a behavioural re-scoring. Both are scripted and reproducible.

What is it actually for? Draft generation for human post-editing. A county health officer writes a cholera advisory in English; the model produces an Ekegusii draft; an Ekegusii speaker corrects it and publishes. That is a large time saving over translating from scratch, and it is not autonomous publication.

What would you do next? In order: collect human evaluation through the correction form; back-translate monolingual Ekegusii text to enlarge the corpus; target institutional vocabulary specifically, possibly with a terminology list; then try the 1.3B model now that the recipe is settled.

10. Terms you should be able to define on the spot

TERM	ONE-LINE DEFINITION
NMT	Neural machine translation — translation by a neural network trained end to end
Encoder–decoder	The encoder reads the source into a representation; the decoder generates the target from it
Token / subword	Text is split into pieces smaller than words, so rare words are built from known parts
Embedding	The vector a token is mapped to; the model's learned representation of that token
Fine-tuning	Continuing training of a pretrained model on new, task-specific data
Transfer learning	Reusing knowledge learned on one task or language to help another
Catastrophic forgetting	A model losing earlier capabilities as it learns something new
Replay	Mixing old data back into new training to prevent that forgetting
Curriculum learning	Ordering training data deliberately, easy-to-hard or general-to-specific
Ablation / control	A run that differs in exactly one way, so a difference can be attributed to that one thing
Beam search	Decoding that keeps the k best partial translations rather than only the single best next token
Agglutinative	A language that builds words by stacking meaningful affixes onto a stem
Low-resource	A language with little digital text or parallel data available
chrF2++	A translation metric over character n-grams plus word uni/bigrams
BLEU	The classic metric, over word n-grams
Held-out set	Data withheld from training and used only to measure performance

11. Suggested talk track

Fifteen slides, roughly 15–18 minutes. The deck is at `docs/Ekegusii_NMT_presentation.pptx` and every slide has speaker notes.

TIME	SLIDES	WHAT YOU ARE DOING
0:00–1:30	1–2	Land the gap. 202 languages, zero Ekegusii, 2.7 million speakers
1:30–3:00	3–4	Objectives and where the data came from
3:00–5:00	5	**The 53.6% vocabulary gap.** Slow down here — it justifies everything after
5:00–7:30	6–7	Adding the token, seeding from Kikuyu, and the three runs
7:30–9:30	8	Why chrF2++. Expect a question; you have the answer
9:30–10:30	9	Setup, briefly. Do not read the table aloud
10:30–12:30	10–12	Results, the negative finding, what the released model achieves
12:30–14:00	13	Limitations, stated before anyone asks
14:00–16:00	14	**Live demo.** Have it open in a tab already — never start it live
16:00–17:00	15	Three takeaways, close on the control

Two rules for the demo. Start it before you walk in, and type a sentence from a domain the model handles well — a health or safety notice — rather than one dense with institution acronyms. If someone in the audience types `KUCCPS` and the output is poor, that is your limitations slide proving itself, and the right response is "yes, that is exactly the 53.6% gap we measured."

12. Where everything lives

WHAT	WHERE
Code, notebooks, data pipeline	`github.com/SamAbr/public-service-anouncement-MT`
Released model	`samuelabrha/nllb-200-600M-ekegusii-mixed` (private)
Baseline	`samuelabrha/nllb-200-600M-ekegusii-stage1` (private)
Curriculum ablation	`samuelabrha/nllb-200-600M-ekegusii-psa` (private)
Live demo	`serve/colab_demo.ipynb` — four cells, free Colab GPU
Deck and banner	`docs/`

The notebook sequence, in the order it runs: `00_setup` → `01_eda` → `02_build_training_data` → `03_extend_tokenizer` → **`train_stages.py`** → `04_evaluate` → `05_inference_and_export`.

Training is a script rather than a notebook because it waits for free VRAM on a shared card, recovers from out-of-memory by halving the batch, and can resume a single stage — none of which survives a kernel restart. It is what produced the released weights.